

CAPGEMINI EXCELLER 2027

Complete Assessment Preparation Guide

Basic → Advanced Prep | Tips & Tricks | Important Notes

Stage	Section	Focus Area
1	English Communication	Listening, Speaking, Reading & Writing
2	Problem Solving	Solve a Coding Task
3	AI-assisted Debugging	Use AI to identify & correct code issues
4	AI-assisted Feature Development	Use AI to build a feature in a given application
5	Prompt Engineering	Write a prompt against a given problem statement
6	Cognitive Assessment	Motion & Grid Challenge, Logical Reasoning, Behavioural Module

Date	What happens
28 September 2026	Section 1 — English Communication Test
5 / 6 / 7 October 2026	Sections 2–6 — Problem Solving, AI Debugging, AI Feature Dev, Prompt Engineering

Personalised for your timeline — built to take you from fundamentals to exam-ready.

Table of Contents

0	How This Assessment Works & 10-Day Master Plan
1	English Communication Test (28 Sep)
2	Problem Solving — Coding Round
3	AI-Assisted Debugging
4	AI-Assisted Feature Development
5	Prompt Engineering
6	Cognitive Assessment (Motion & Grid, Logical Reasoning, Behavioural)
7	Universal Tips, Tricks & Exam-Day Strategy
8	Final Checklist — Night Before & Day Of

0 How This Assessment Works & 10-Day Master Plan

Capgemini's **Exceller 2027** hiring journey tests you across six back-to-back stages. Section 1 (English Communication) is on **28 September**, and Sections 2–6 happen together in **one single sitting on 5, 6 or 7 October**. That second sitting is long and mentally demanding — treat it like an exam marathon, not five separate quizzes.

Why the single-sitting round matters most

- You'll move from **coding** → **debugging** → **building a feature with AI** → **prompting** → **cognitive puzzles** without a long break. Stamina and time-management matter as much as skill.
- Sections 3–5 (AI-assisted Debugging, AI-assisted Feature Development, Prompt Engineering) are the newest and most unfamiliar for most candidates — prioritise these in your prep.
- The Cognitive Assessment (Motion & Grid, Logical Reasoning, Behavioural) is typically timed per-question — speed and pattern recognition beat perfectionism.

Your 10-Day Master Plan (25 Sep → 7 Oct)

Day	Focus	What to do
Day 1 (25 Sep)	English foundations	Grammar refresh, read 20 min news aloud, record yourself speaking 2 min on a random topic.
Day 2 (26 Sep)	English — Listening & Writing	Practice dictation/listening exercises; write 2 short emails/essays; learn 15 professional vocabulary words.
Day 3 (27 Sep)	English — Mock + Revise	Take 1 full mock English test under timed conditions. Review mistakes. Sleep well.
Day 4 (28 Sep)	■ ENGLISH TEST DAY	Warm up voice 10 min before. Stay calm, speak naturally, don't rush.
Day 5 (29 Sep)	Coding fundamentals	Revise arrays, strings, loops, basic DSA. Solve 5 easy problems on paper/IDE.
Day 6 (30 Sep)	Coding practice	Solve 5–8 medium problems (arrays, strings, hashmaps). Time yourself (20–25 min each).
Day 7 (1 Oct)	AI-assisted Debugging + Feature Dev	Practice reading buggy code, using AI tools (Copilot/ChatGPT-style) to fix & extend code.
Day 8 (2 Oct)	Prompt Engineering	Learn prompt frameworks (Section 5). Practice writing 10 prompts for varied problem statements.
Day 9 (3–4 Oct)	Cognitive Assessment + Full Mock	Practice logical reasoning & pattern/grid puzzles daily (20 min). Do one full timed mock of Sections 2–6.
Day 10 (5–7 Oct)	■ MAIN TEST DAY	Light revision only. Review notes, sleep 7–8 hrs, arrive/login early, stay composed.

■ Golden Rule

Don't try to master everything equally. Spend the most hours on **Prompt Engineering** and **AI-assisted Debugging/Feature Development** since they're the least 'traditional' sections and where preparation gives you the biggest edge over other candidates.

1 English Communication Test (28 Sep)

This section evaluates **Listening, Speaking, Reading and Writing** — usually via an AI/voice-based platform (similar to Versant/AMCAT English tests). It checks fluency, grammar, pronunciation, and comprehension, not your accent.

1.1 Format Breakdown (Typical Structure)

Skill	What's tested	How to prepare
Listening	Understand short audio clips, answer questions or repeat sentences	Listen to BBC/CNN news or podcasts daily; practice shadowing (repeat what you hear immediately)
Speaking	Read aloud, repeat sentences, describe a picture, answer open questions, give an opinion in 60–90 sec	Record yourself daily; speak on random topics without pausing to think; focus on clear pacing not speed
Reading	Read a passage aloud / comprehension MCQs	Read newspaper editorials aloud 15 min/day; note tricky words and their pronunciation
Writing	Write an email/essay/summary on a given topic, grammar correction, sentence construction	Practice 150–200 word emails on workplace topics; learn common grammar rules (tenses, articles, prepositions)

1.2 Question Types You'll Likely See

- **Read Aloud:** A sentence/paragraph appears — read it clearly and naturally within a time limit.
- **Repeat Sentence:** Hear a sentence once, repeat it exactly. Tests memory + pronunciation.
- **Describe an Image/Situation:** Speak for 30–60 seconds describing what you see or think.
- **Answer Short Question:** Respond to a question in one word or short phrase quickly (tests reflexes).
- **Retell/Summarize:** Listen to a short passage, then summarize it in your own words.
- **Email/Essay Writing:** Write a structured response (greeting, body, closing) to a workplace scenario.
- **Grammar/Error Spotting:** Identify or correct the incorrect part of a sentence.

1.3 Tips & Tricks

- **Speak at a natural, steady pace** — not too fast (sounds rushed/unclear to the AI scorer) or too slow (sounds hesitant).
- **Don't leave silence.** If unsure, keep speaking — a confident, imperfect answer scores better than silence or excessive 'umm's.
- Use your **full response time** in speaking tasks — very short answers lower fluency/content scores.
- For Read Aloud: **glance ahead** at the whole sentence before you start reading so you don't stumble mid-way.
- In writing tasks, follow a simple structure: **Greeting** → **2–3 clear body points** → **Polite closing**. Keep sentences short and grammatically clean rather than complex and risky.
- Avoid slang/contractions in writing (use "do not" instead of "don't") — sounds more professional.
- Test your **mic and headset** beforehand; sit in a quiet room; speak directly into the mic at consistent volume.

■ Common Mistakes to Avoid

Mumbling or trailing off at the end of sentences — AI scoring systems penalise low clarity at sentence-end.

Overthinking grammar mid-speech and pausing too long — this hurts fluency score more than a small grammar slip.

Copy-pasting or reading directly off notes in writing tasks if the platform checks originality/typing patterns.

Background noise, fans, or talking during recording — disqualifies or lowers audio clarity score.

1.4 Important Notes

- Most AI English tests score on: **Fluency, Pronunciation, Content/Relevance, Vocabulary, Grammar** — a balanced answer beats a perfect-but-short one.
- Practice daily out loud — even 10 minutes of reading news aloud significantly improves fluency scores within a week.
- Keep a small personal vocabulary list of 20–30 professional/business English words (e.g., *facilitate, collaborate, streamline, prioritize, escalate*) to naturally use in speaking/writing answers.

2 Problem Solving — Coding Round

This is a hands-on coding task, typically solved in an online IDE (often language-agnostic: C/C++/Java/Python/JS). It tests your core programming and logical thinking, usually within a compiler that runs your code against test cases.

2.1 Core Topics to Revise

- **Arrays & Strings:** traversal, searching, sorting, two-pointer technique, sliding window
- **Basic Data Structures:** stacks, queues, linked lists, hashmaps/dictionaries, sets
- **Loops & Conditionals:** nested loops, edge case handling (empty input, negative numbers, duplicates)
- **Recursion & Basic DP:** factorial, Fibonacci, simple memoization
- **Time & Space Complexity:** know Big-O basics ($O(n)$, $O(n \log n)$, $O(n^2)$) to write efficient solutions
- **String Manipulation:** palindrome checks, anagrams, substring problems, string reversal
- **Math/Logic Basics:** prime numbers, GCD/LCM, number patterns

2.2 Approach Strategy (Step-by-Step)

- **Step 1 — Read twice:** Read the full problem statement twice before coding. Note input/output format and constraints.
- **Step 2 — Identify pattern:** Is it array traversal? Searching? String parsing? Map to a known pattern.
- **Step 3 — Write pseudocode/dry run** on paper or comments first for 1–2 minutes — saves debugging time later.
- **Step 4 — Code incrementally:** Write and test small chunks rather than the whole solution at once.
- **Step 5 — Test edge cases:** empty input, single element, largest possible input, duplicate/negative values.
- **Step 6 — Optimize only if time permits** — a correct working (even brute-force) solution beats an incomplete optimized one.

2.3 Tips & Tricks

- Use **meaningful variable names** — improves readability if code is manually reviewed.
- Always **handle edge cases** explicitly (empty arrays, 0, negative numbers) — most test cases fail here.
- If stuck, code the **brute-force solution first**, get partial test cases passing, then optimize.
- Keep an eye on the **timer** — don't spend more than 60% of allotted time on one problem if there are multiple.
- Comment your logic briefly — helps you and any reviewer understand your thought process.
- Practice on **HackerRank / LeetCode (Easy-Medium)** in the days before — aim for 15–20 problems covering arrays, strings, and hashmaps.

■ Common Mistakes

Jumping straight into code without understanding constraints (e.g., missing that input can be negative).

Ignoring compiler/syntax errors and not testing before final submission.

Spending too long perfecting one problem and running out of time for others.

3 AI-Assisted Debugging

You'll be given code with existing bugs/issues and expected to **use an AI assistant** (chat-based coding assistant) to identify and correct them. This tests both your debugging skill AND your ability to collaborate effectively with AI — you are being evaluated on judgment, not just the AI's output.

3.1 What's Actually Being Tested

- Can you **correctly diagnose** what's wrong before/while using AI (not just blindly copy-paste AI suggestions)?
- Can you write **clear, specific prompts** to the AI describing the bug/symptom?
- Can you **verify and validate** the AI's fix — does it actually solve the root cause, or just suppress the symptom?
- Do you understand the corrected code well enough to explain it if asked?

3.2 Step-by-Step Debugging Workflow

- **1. Reproduce first:** Run the code, note the exact error message or wrong output.
- **2. Read the code manually** for 1–2 minutes — form your own hypothesis about the bug (off-by-one? wrong condition? type mismatch? null/undefined?).
- **3. Ask AI with context**, not just "fix this": include the code, the error message, expected vs actual output, and your hypothesis.
- **4. Evaluate the AI's suggestion** — does it match the root cause, or does it just patch a symptom? Re-run to confirm the fix.
- **5. Check for side-effects** — did the fix break another part of the logic? Test multiple scenarios.
- **6. Summarize** in a comment or your own words: what was wrong and why the fix works (this may be checked).

3.3 High-Value Prompt Patterns for Debugging

Situation	Prompt pattern to use
Code throws an error	"This code throws [exact error]. Here's the code: [paste]. Expected behaviour: [X]. What is causing this and how do I fix it?"
Wrong output, no error	"This function should return [expected] for input [X] but returns [actual]. Here is the code: [paste]. Identify the logical bug."
Not sure where the bug is	"Review this code for logical errors, edge-case issues, and off-by-one mistakes: [paste code]"
After getting a fix	"Explain why this fix resolves the issue, and check if it could break any other case."

3.4 Tips & Tricks

- Always give the AI the **exact error text/output** — vague prompts like "this doesn't work" produce vague, unhelpful fixes.
- Don't accept the first AI answer blindly — **trace through the logic yourself** before finalizing it.
- If the AI gives a fix you don't understand, **ask it to explain** — you may be asked to justify your solution.
- Common bug categories to know by heart: off-by-one errors, incorrect loop bounds, wrong comparison operators (= vs ==), type mismatches, null/undefined handling, incorrect variable scope.

■ Common Mistakes

Copy-pasting AI's fix without testing it against the given scenario.

Writing prompts with no code/context — wastes turns and produces generic advice.

Not double-checking that the fix doesn't introduce a new bug elsewhere.

4 AI-Assisted Feature Development

You'll be given an existing application/codebase and asked to **use AI effectively to develop a new feature**. This tests whether you can break a feature request into implementable steps and direct an AI tool to build it correctly within an existing codebase — a very real day-to-day skill at Capgemini.

4.1 Step-by-Step Approach

- **1. Understand the existing app first:** Skim the folder structure, key files, and how data flows before writing any prompt. Don't dive in blind.
- **2. Break the feature into small steps:** e.g., "add a search bar" → (a) UI input element, (b) filter/search logic, (c) connect to existing data, (d) handle empty/no-result state.
- **3. Prompt AI per step**, not the whole feature at once — smaller, scoped prompts give more reliable, reviewable code.
- **4. Match existing code style** (naming conventions, indentation, patterns already used in the file) — mention this to the AI explicitly.
- **5. Test as you go** — after each AI-generated chunk, verify it runs and integrates before moving to the next step.
- **6. Handle edge cases:** empty inputs, no results, invalid data — good candidates always ask the AI to cover these.

4.2 Prompt Framework for Feature Requests

■ Use this structure

Context — "Here is the existing code/file: [paste relevant portion]"

Goal — "I need to add [specific feature] that does [specific behaviour]"

Constraints — "It must follow the existing naming/style, not break [existing functionality], and handle [edge case]"

Ask — "Write the code for this, and explain what you changed"

4.3 Tips & Tricks

- Think like a developer, not just a prompt-writer: **plan the feature on paper first** (what changes, where, why) before prompting.
- Keep the AI's suggestions **modular** — one function/component per prompt is easier to verify than one giant change.
- Always **re-read generated code** — you may be asked to explain your implementation choices.
- If the AI's first attempt is close but not quite right, **iterate with specific feedback** ("this works but doesn't handle empty input — fix that") rather than starting over.
- Mention the **tech stack/language** explicitly in your first prompt so the AI doesn't guess wrong.

■ Common Mistakes

Asking for the entire feature in one giant vague prompt — leads to code that doesn't fit the existing app.

Not checking whether the new feature breaks existing functionality.

Ignoring the app's existing patterns/style, producing inconsistent code.

5 Prompt Engineering

You'll be given a problem statement and asked to **write an effective prompt** to solve it using AI — this section tests your ability to communicate precisely with an AI model, not your coding or the AI's raw ability.

5.1 The Core Prompt Framework — "C.R.A.F.T."

Element	What to include	Example
Context	Background info the AI needs	"You are helping analyze customer support tickets for an e-commerce company."
Role	What persona/expertise the AI should adopt	"Act as a senior data analyst."
Action/Task	The specific, clear task to perform	"Categorize the following 10 tickets into Billing, Technical, or General."
Format	Exact output format expected	"Return the result as a table with columns: Ticket ID, Category, Confidence."
Tone/Constraints	Any limits — length, style, what to avoid	"Keep explanations under 2 lines. Do not include assumptions not stated in the ticket."

5.2 Writing High-Quality Prompts — Key Principles

- **Be specific, not vague:** "Summarize this" is weak. "Summarize this in 3 bullet points focused on cost impact" is strong.
- **State the output format explicitly:** table, JSON, bullet list, code block, word count — don't leave it to guesswork.
- **Give examples when possible** (few-shot prompting) — showing the AI one example of desired output dramatically improves accuracy.
- **Break complex tasks into steps** within the prompt ("First do X, then do Y, then Z") rather than one dense sentence.
- **Set constraints** — word limits, what NOT to include, tone (formal/casual) — this shows precision and control.
- **Ask AI to show reasoning** when the task is analytical: "explain your reasoning before giving the final answer."

5.3 Worked Example

■ Weak vs Strong Prompt

Weak: "Write code to sort a list."

Strong: "Act as a Python developer. Write a function `sort_list(arr)` that sorts a list of integers in ascending order using an efficient algorithm ($O(n \log n)$ or better). Include a docstring, handle empty lists, and return the sorted list. Do not use built-in `sort()` — implement the algorithm manually. Output only the code, no explanation."

5.4 Tips & Tricks

- Always re-read the **given problem statement** carefully — the prompt you write must directly address every requirement mentioned, not just the general topic.
- If the problem statement gives constraints (e.g., "must be under 100 words", "must be in JSON"), **bake those exact constraints into your prompt** — this is often what's graded.

- Avoid overly long, rambling prompts — **clarity beats length**. Use short, direct sentences.
- When unsure, structure your prompt in labelled parts (Context: / Task: / Format: / Constraints:) — this is easy to grade and shows structured thinking.
- Practice writing 8–10 prompts for varied scenarios (data analysis, code generation, content writing, classification) before the test so the structure becomes automatic.

■ Common Mistakes

Writing a prompt that's technically fine but ignores specific details in the given problem statement.

Being too vague about the desired output format, leading to unusable AI responses.

Forgetting to specify constraints/tone/length, resulting in bloated or off-target output.

6 Cognitive Assessment

This section has three parts: **Motion & Grid Challenge**, **Logical Reasoning**, and a **Behavioural Module**. It's usually timed per question and adaptive — speed and pattern recognition matter as much as accuracy.

6.1 Motion & Grid Challenge

Typically involves spatial reasoning: predicting how shapes/objects move, rotate, or fill within a grid; identifying the next step in a visual sequence, or completing a grid pattern.

- **Practice type:** similar to non-verbal reasoning tests — shape rotation, mirror images, grid pattern completion, sequence prediction.
- **Strategy:** Look at one attribute at a time (shape → size → shading → rotation → position) instead of trying to see the whole pattern at once.
- **Speed tip:** Eliminate 2 obviously wrong options first — narrows choice fast under time pressure.
- **Practice source:** Search "non-verbal reasoning practice tests" or "abstract reasoning tests" (common in aptitude prep sites) — do 15–20 questions/day for 3 days before the test.

6.2 Logical Reasoning

- **Common question types:** syllogisms, blood relations, coding-decoding, number/letter series, seating arrangements, direction sense, statement-conclusion.
- **Strategy:** For series questions, check the difference/ratio pattern first (arithmetic, then geometric, then alternate patterns).
- **Strategy:** For coding-decoding, map each letter to its position/shift systematically rather than guessing.
- **Strategy:** For syllogisms, always draw quick Venn diagrams — don't solve purely in your head.
- **Practice:** Solve at least 30–40 mixed logical reasoning questions in the 2 days before your test, timing yourself at ~45–60 sec/question.

6.3 Behavioural Module

This assesses **workplace personality traits** — usually a situational judgment test (SJT) or self-report questionnaire (e.g., how you'd react to a conflict, a tight deadline, unclear instructions, or team disagreement). There's no single "correct" answer, but responses are scored against traits Capgemini values.

- **Be consistent, not extreme:** avoid always picking the most extreme option ("always"/"never") — balanced, realistic answers score better than performative ones.
- **Answer as your genuine best professional self** — inconsistent answers (contradicting yourself across similar questions) can flag as unreliable.
- **Traits typically valued:** collaboration, ownership/accountability, adaptability, client-centric thinking, integrity, proactive communication — keep these in mind when choosing between similar options.
- **Don't overthink each question** — SJTs are usually timed; go with your genuine first instinct rather than second-guessing.

■ Quick Practice Routine (3 days before)

Day -3: 20 min grid/spatial reasoning + 20 min logical reasoning practice.

Day -2: 20 min number/letter series + 20 min syllogisms/blood relations.

Day -1: One full timed mock cognitive test (30–40 questions) to build speed and stamina.

7 Universal Tips, Tricks & Exam-Day Strategy

7.1 Time Management Across Sections

- Before you start Sections 2–6, quickly check the **total time and number of sections** shown on screen — mentally allocate a time budget per section.
- If a section has multiple questions, follow the rule: **don't spend more than 1.5x the average time** on any single question — flag/skip and return if allowed.
- Keep 5 minutes buffer at the end of each section to **review and submit** — don't let the timer auto-submit unreviewed work.

7.2 Technical Setup (Do this 1 day before)

- Test your **internet connection, webcam, and microphone** — most proctored tests require both camera and mic access.
- Use a **laptop/desktop** (not mobile) with a stable wired connection if possible; close all other tabs/apps.
- Sit in a **well-lit, quiet, plain-background room** — proctoring AI may flag excessive movement or noise.
- Keep your **ID proof** and any required documents ready beside you, as instructed in your invite email.
- Charge your device fully or stay plugged in; keep a backup power/internet option if possible.

7.3 Mindset & Exam-Day Strategy

- **Start with the section you're most confident in** if the platform allows choosing order — builds momentum.
- If you get stuck, **don't panic-freeze** — attempt a partial/logical answer rather than leaving it blank; partial credit is common in coding/AI tasks.
- Read every instruction and problem statement **twice** before answering — many marks are lost to misreading, not lack of skill.
- Stay hydrated, take a 5–10 min break before the long sitting on 5/6/7 Oct if there's a gap between sections.
- Treat AI as a **collaborator, not a crutch** — across Sections 3–5, examiners are specifically checking that YOU are directing the AI with judgment, not just accepting whatever it outputs.

Quick Reference — Do's and Don'ts

■ Do	■ Don't
Read the full problem/prompt before acting	Start coding or prompting immediately without understanding requirements
Verify every AI output before accepting it	Blindly copy-paste AI suggestions
Keep prompts specific, structured, and example-driven	Write vague, one-line prompts
Manage time per section with a mental budget	Spend excessive time perfecting one question
Speak naturally and use full response time in English test	Rush answers or leave long silences
Test your setup (mic/cam/net) the day before	Wait until test time to check your tech setup

8 Final Checklist — Night Before & Day Of

The Night Before

- ■ Light revision only — skim your notes for Prompt Engineering & AI-debugging patterns (Sections 3–5), don't cram new topics.
- ■ Test laptop, webcam, mic, and internet connection one final time.
- ■ Keep your ID proof, admit card/invite email, and any required documents ready.
- ■ Set 2 alarms; plan to log in **15–20 minutes early**.
- ■ Sleep at least 7–8 hours — cognitive and language performance drops sharply with poor sleep.

On the Day

- ■ Eat a light meal; avoid heavy food right before the test.
- ■ Sit in a quiet, well-lit room with a plain background.
- ■ Keep water nearby; silence your phone completely.
- ■ Do a 5-minute warm-up: read a paragraph aloud (for English test) or solve 1 quick logic puzzle (for the 5/6/7 Oct sitting) to get your brain active.
- ■ Breathe, stay composed between sections — you've prepared for this systematically.

■ You've Got This

You now have a structured plan covering every stage of the Capgemini Exceller 2027 journey — from English fluency to AI-assisted coding to cognitive speed. Follow the 10-day plan, focus extra hours on Sections 3–5 (AI Debugging, Feature Development, Prompt Engineering) since they carry the most differentiation, and walk in with a calm, structured approach. Good luck!